

13/7/26

Python Programming Assignment - 24

1) What is a lambda function in Python? How it is different from a normal function?

Ans :-

A lambda function is a small anonymous function in Python. Anonymous function means a function without a name. Lambda function is used to create small, throwaway functions without formally defining them using `def`.

A normal function declared with `def` must be assigned a name, can contain multiple lines of code including loops and complex conditional statements, & requires an explicit `return` keyword to output a value. In contrast, a lambda function is strictly limited to a single expression, contain multiple statements or variable assignments inside its body, and automatically returns the evaluated result without you ever typing the `return` keyword. Essentially, while `def` is used to create robust, reusable blocks of logic, lambda is meant for writing quick, simple operations that are only needed at the exact moment they are created.

2) What are the limitations of lambda functions?

Ans :-

The most significant limitations of a lambda function in Python is its strict restriction to a single expression. Unlike normal functions defined with `def`, you cannot include multiple statements, variable assignments, or standard looping structures like `for` and `while` within a lambda. While you can use a basic ternary operator (an inline `if-else`), you cannot write full, multi-line

conditional blocks, which makes lambda functions entirely unsuitable for handling complex logic or extensive data manipulation. Furthermore, you are not allowed to use standard Python statements such as return, pass, assert, or raise inside the body of a lambda.

Another major drawback is the negative impact on readability and debugging. Because lambda functions are anonymous, they lack a formal identifier. If an error occurs inside a lambda, the system traceback will simply point to <lambda> rather than a descriptive function name, making it harder to track down the source of the issue. Additionally, lambda functions do not support type annotations or docstrings, making it difficult for other developers to understand the expected inputs and outputs without external context. Ultimately, if the logic inside a lambda starts getting long or complicated, it quickly becomes unreadable and defeats its own purpose, signaling that it should be rewritten as a standard, named function.

3) Explain the working of the map() function with an example.

Ans: The map() function in Python is a built-in higher-order function that allows you to process and transform all the items in an iterable, such as a list or a tuple, without writing an explicit for loop. It takes two primary arguments: a function and an iterable. When executed, map() passes each element from the iterable into the provided function, one by one, and collects the returned values. This provides a clean, memory-efficient way to apply a uniform operation - like converting string formats,

DATE / /

extracting object properties, or performing mathematical calculations - across an entire dataset.

Because `map()` is designed to be highly efficient, it does not immediately return a newly populated list in memory. Instead, it returns a "map object", which is an iterator that computes the transformed values lazily, only when they are explicitly requested. To view, print, or store the results as a standard collection, you must convert this map object using a constructor like `list()` or `tuple()`. Furthermore, `map()` is exceptionally useful when paired with lambda functions, as it allows you to define the transformation logic directly inline, keeping your code concise and preventing the need to write standalone functions for simple tasks.

4) How does `map()` differ from using a for loop?

Ans:

- i) **Memory Efficiency (Lazy Evaluation):** A for loop that builds a new list loads all elements into memory at once. `map()` returns an iterator, meaning it generates items one by one on the fly, which is vastly more memory-efficient for massive datasets.
- ii) **Immutability and State:** A for loop explicitly mutates state (eg, appending to an external list variable). `map()` promotes a functional style by avoiding side effects; it simply takes an input and produces an output.
- iii) **Performance:** Under the hood, `map()` is written in C and optimized for internal iteration. For simple built-in

functions, `map()` can execute slightly faster than a standard Python for loop.

iv) Readability : `map()` expresses the intent for the code (transforming data), whereas a for loop expresses the mechanics (how the transformation happens step-by-step).

5) What is the `filter()` function and when should it be used?

Ans: The `filter()` function constructs an iterator from elements of an iterable for which the passed function returns True.

- When to use it: You use it when you have a large dataset and need to extract a smaller subset of elements that satisfy a specific condition (eg, extracting all even numbers from a list, or dropping all None values).

6) Difference between `map()` and `filter()`.

Ans: i) ~~Map~~ `map()`

- purpose: Transforms every item in an iterable.
- Output size: Always equal to the input size (1-to-1).
- Function Requirement: Function can return any value.

ii) `filter()`

- purpose: Selects / filters items based on a condition.
- Output size: Less than or equal to the input size.
- Function Requirement: Function must return a Boolean (True/False).

7) What is `reduce()`? Why is it not a built-in function in Python?

Ans: `reduce()` applies a function of two arguments cumulatively to the items of an iterable, from left to right, reducing the sequence to a single final value.

Why it's not built-inⁱⁿ: In Python 2, `reduce()` was built-in. In Python 3, Guido van Rossum (Python's creator) moved it to the `functools` module. He argued that `reduce()` was often abused to write highly unreadable code, and that 99% of the time, a simple for loop with an accumulator variable is much easier for humans to read and understand.

8) Explain how `reduce()` works internally.

Ans:

- i) It takes the first two elements from the iterable and passes them to the function.
- ii) It takes the result of that function and the next element in the iterable, and passes them both to the function again.
- iii) It repeats this process sequentially until the iterable is exhausted, returning one final accumulated value.

9) Can lambda functions be used with map, filter, and reduce together? Explain.

Ans: Yes, absolutely. Because, map and filter both take iterables and return iterables, they can be chained together, and lambda functions are perfect for passing the quick, inline logic needed for each step.

Example: Summing the squares of only the even numbers in a list.

```
from functools import reduce
```

```
nums = [1, 2, 3, 4, 5, 6]
```

```
Ret = reduce(lambda x, y: x+y, map(lambda x: x**2, filter(lambda x: x%2==0, nums)))
```

10) When should functional programming be preferred over procedural style in Python?

Ans: • Prefer Functional (map/filter/reduce):

When you are applying simple data transformations, doing mathematical computations, or want to avoid "side effects". It often results in cleaner, more concise code.

• Prefer Procedural:

When the logic is complex, involves managing state (like UI updates or database connections), requires extensive I/O operations, or when chaining functional methods together becomes so dense that it hurts code readability.